

Introduction to MongoDB

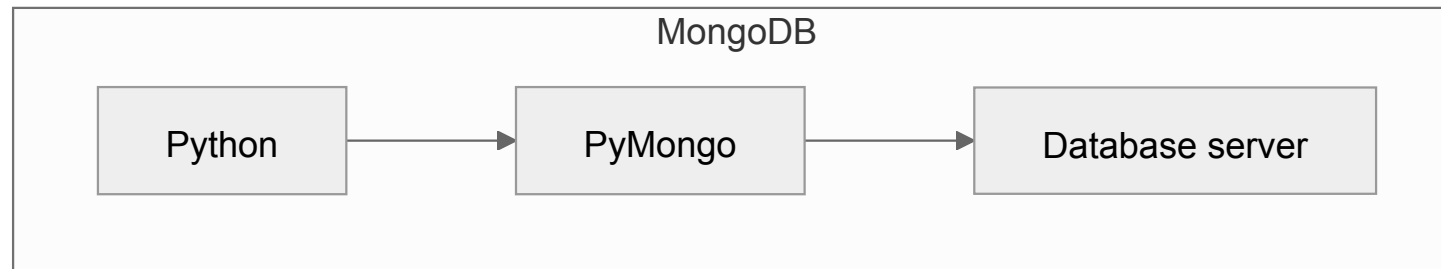
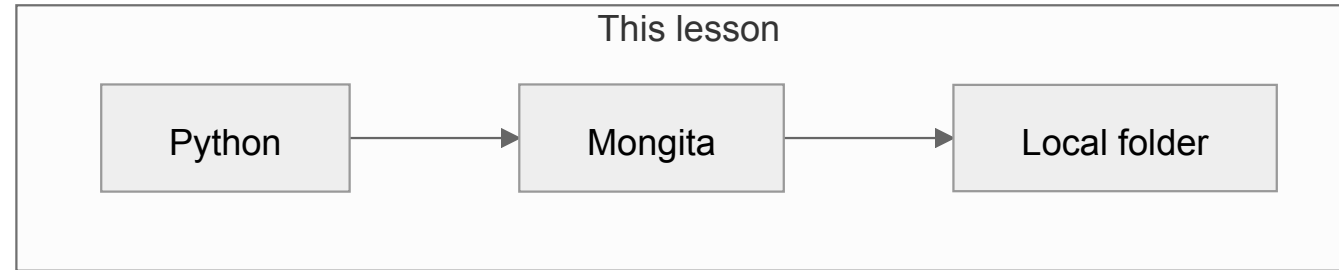
Documents and collections with Mongita

Gregory S. DeLozier, PhD

The Session

- Store two documents
- Query and update fields
- Consume a cursor
- Delete, reinsert, and reopen

Where the Database Runs



Mongita runs inside the Python program.

Two Documents

```
{"name": "Meercat", "does_not_eat": "Snakes"}
```

```
{"name": "Yellow mongoose", "eats": "Termites"}
```

Must every document have the same fields?

Database, Collection, Document

Level	Our example
Database	hello_world_db
Collection	mongoose_collection
Document	One mongoose record
Field	name or eats

Open the Local Collection

```
from lesson import connect  
  
client, mongoose_collection = connect()
```

Storage: code/data/mongita

Keep generated data out of Git.

Insert Documents

```
mongoose_collection.insert_many([  
    {"name": "Meercat", "does_not_eat": "Snakes"},  
    {"name": "Yellow mongoose", "eats": "Termites"},  
])
```

Setup already ran this insertion.

Find One Document

```
yellow = mongoose_collection.find_one(  
    {"eats": "Termites"}  
)  
print(yellow["name"])  
print(yellow["_id"])
```

Result: a dictionary, or **None**

Filters

Filter	Selects
<code>{}</code>	All documents
<code>{"name": "Meercat"}</code>	This name
<code>{"weight": {"\$gt": 1}}</code>	Numeric weight greater than 1

Add a Field with an Update

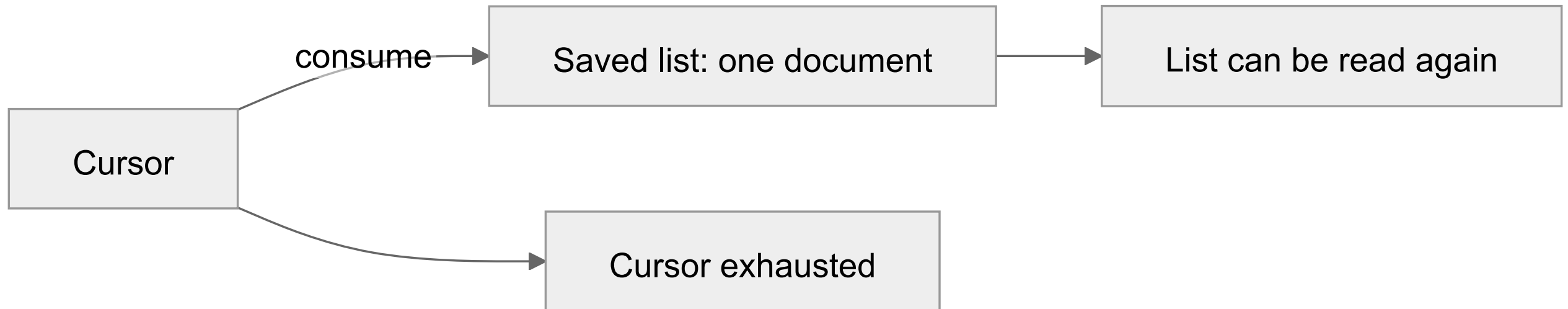
```
result = mongoose_collection.update_one(  
    {"name": "Meercat"},  
    {"$set": {"weight": 2}},  
)
```

Filter first. Change second.

Which document now has a weight?

Query Results Through a Cursor

```
cursor = mongoose_collection.find(  
    {"weight": {"$gt": 1}}  
)  
mongooses = list(cursor)
```



What Does Each Line Print?

```
print(len(mongooses))  
print(list(cursor))  
print(len(mongooses))
```

1, then [], then 1

No Matching Document

```
query = {"weight": {"$gt": 300}}  
  
print(mongoose_collection.find_one(query))  
print(list(mongoose_collection.find(query)))
```

None versus []

An Identifier in a Web Address

```
from bson.objectid import ObjectId

identifier = str(yellow["_id"])
found = mongoose_collection.find_one(
    {"_id": ObjectId(identifier)}
)
```

Convert back to the stored value type.

Delete the Record We Read

```
old = mongoose_collection.find_one(  
    {"name": "Meercat"}  
)  
result = mongoose_collection.delete_one(  
    {"_id": old["_id"]}  
)
```

`result.deleted_count` is **1**

Reinsert the Same Name

```
mongoose_collection.insert_one(
    {"name": "Meercat", "does_not_eat": "Snakes"}
)
```

Before deletion	After reinsertion
Old _id	New _id
weight is 2	No weight field

Exit and Reopen

```
client.close()  
exit()
```

Start Python again and open the same collection.

Which values survive?

The Application Connection



`get_pet()` and `update_pet()` can keep their jobs.

Explore the Stored Data

Add three records; give only two a weight.

Update one. Repeat your query. Reopen Python.

Explain the results from the documents you find.